**Министерство науки и высшего образования Российской Федерации**

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**ITMO University**

**ЛАБОРАТОРНАЯ 2**

**По дисциплине** Контейнеризация и оркестрация приложений

**Тема работы** Знакомство с экосистемой Docker: от базовых команд к диагностике сервиса.

**Обучающийся** Ямалеева Азалия Фиразовна

**Факультет** Прикладной информатики

**Группа** К3320

**Направление подготовки** 11.03.02 Инфокоммуникационные технологии и системы связи

**Образовательная программа** Программирование в инфокоммуникационных системах

| | | | |
|---|---|---|---|
| **Обучающийся** | _____ | _____ | Ямалеева.А.Ф |
| | (дата) | (подпись) | (Ф.И.О.) |
| **Руководитель** | _____ | _____ | Аминов Н.С. |
| | (дата) | (подпись) | (Ф.И.О.) |

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ ..... 4
1 СОЗДАНИЕ БАЗОВОГО ОБРАЗА ..... 5
1.1 Подготовка и базовый Dockerfile ..... 5
1.2 Сборка образа ..... 5
1.3 Запуск контейнера и проверка работы ..... 6
2 ОПТИМИЗАЦИЯ ОБРАЗА ..... 7
2.1 Оптимизированный Dockerfile ..... 7
2.2 Сборка и сравнение размеров ..... 7
2.3 Multi-stage сборка ..... 8
3 ЧИСТОТА ОБРАЗА ..... 10
3.1 Создание .dockerignore ..... 10
3.2 Пересборка образа ..... 10
4 ПРОВЕРКА ЗДОРОВЬЯ ..... 12
4.1 Добавление healthcheck ..... 12
4.2 Сборка и проверка статуса ..... 13
5 ФИНАЛЬНЫЙ АРТЕФАКТ И ТЕГИРОВАНИЕ ..... 14
ЗАКЛЮЧЕНИЕ ..... 15

# ВВЕДЕНИЕ

Цель работы – научиться собирать образы Docker из Dockerfile, оптимизировать их размер, настраивать .dockerignore и проверку здоровья (healthcheck), а также тегировать образы.

Исходное приложение: https://github.com/FilBTi/assist-py. Файлы работы размещены в репозитории https://github.com/shimanorian/Containers.

Персональные данные:

| Параметр | Значение |
|---|---|
| Номер ИСУ | 468183 |
| Персональный порт | 5513 |
| Персональный префикс | yamaleeva |
| Секретный ключ | secret83yamaleeva |

# 1 СОЗДАНИЕ БАЗОВОГО ОБРАЗА

## 1.1 Подготовка и базовый Dockerfile

Репозиторий приложения склонирован командой git clone, локальная папка привязана к репозиторию на GitHub (рисунок 1). Базовый Dockerfile создан на основе образа python:3.11-slim. В нём заданы рабочая директория /app, установка зависимостей из requirements.txt, копирование кода приложения, порт 5513 и команда запуска uvicorn (рисунок 2).

```
PS C:\Users\yujau> function prompt { "yamaleeva@$env:COMPUTERNAME> " }
yamaleeva@ALOVINNE> git config --global user.name "yamaleeva"
yamaleeva@ALOVINNE> git config --global user.email "yujausk@gmail.com"
yamaleeva@ALOVINNE> git clone https://github.com/FilBTi/assist-py
Cloning into 'assist-py'...
remote: Enumerating objects: 29, done.
remote: Counting objects: 100% (29/29), done.
remote: Compressing objects: 100% (21/21), done.
remote: Total 29 (delta 7), reused 25 (delta 5), pack-reused 0 (from 0)
Receiving objects: 100% (29/29), 19.36 KiB | 295.00 KiB/s, done.
Resolving deltas: 100% (7/7), done.
yamaleeva@ALOVINNE> cd assist-py
yamaleeva@ALOVINNE> git remote set-url origin https://github.com/shimanorian/Containers.git
yamaleeva@ALOVINNE> git branch -M main
```

Рисунок 1 – Клонирование репозитория и настройка git

```
yamaleeva@ALOVINNE> $u = New-Object System.Text.UTF8Encoding $false
yamaleeva@ALOVINNE> [System.IO.File]::WriteAllText("$PWD\Dockerfile", @'
>> FROM python:3.11-slim
>> LABEL maintainer="yamaleeva@lab.local"
>> LABEL version="1.0"
>> WORKDIR /app
>> COPY requirements.txt .
>> RUN pip install -r requirements.txt
>> COPY app/ ./app/
>> EXPOSE 5513
>> CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "5513"]
>> '@, $u)
```

Рисунок 2 – Создание базового Dockerfile

## 1.2 Сборка образа

Образ собран командой docker build -t yamaleeva-assistant:v1 . (рисунок 3).

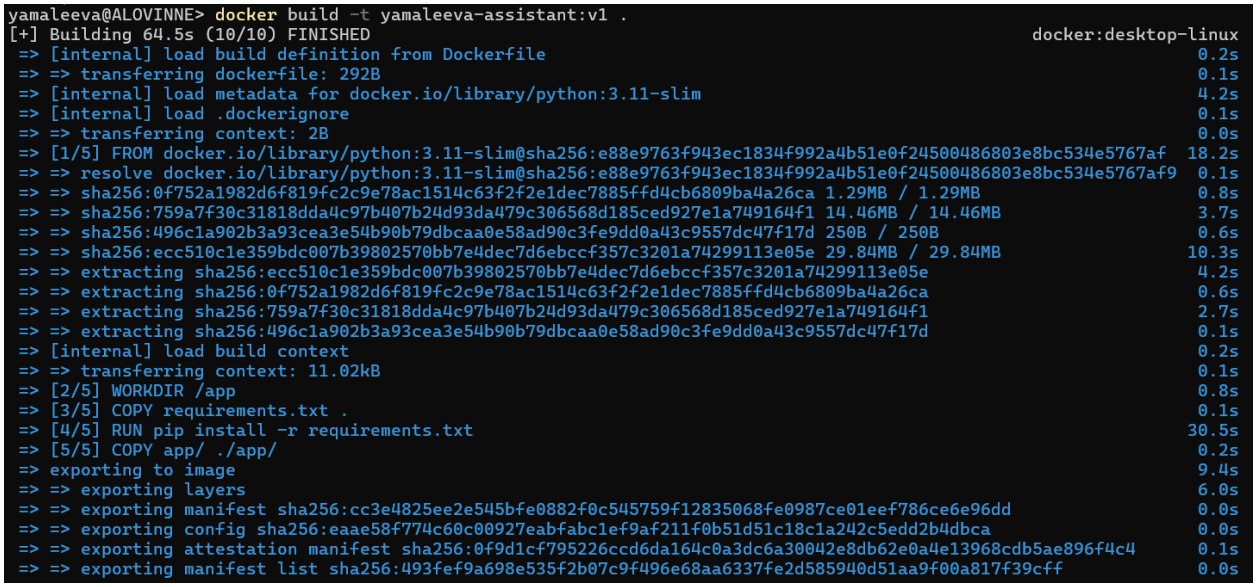



Рисунок 3 – Сборка образа yamaleeva-assistant:v1

### 1.3 Запуск контейнера и проверка работы

Контейнер запущен в фоне с переменными окружения APP_OWNER и APP_SECRET, порт 5513 проброшен на хост. Проверены эндпоинты: главная страница вернула приветствие, /me вернул владельца yamaleeva, заметка добавлена через POST /notes и появилась в списке, /secret/secret83yamaleeva вернул флаг LAB2_SUCCESS_YAMALEEVA (рисунок 4).

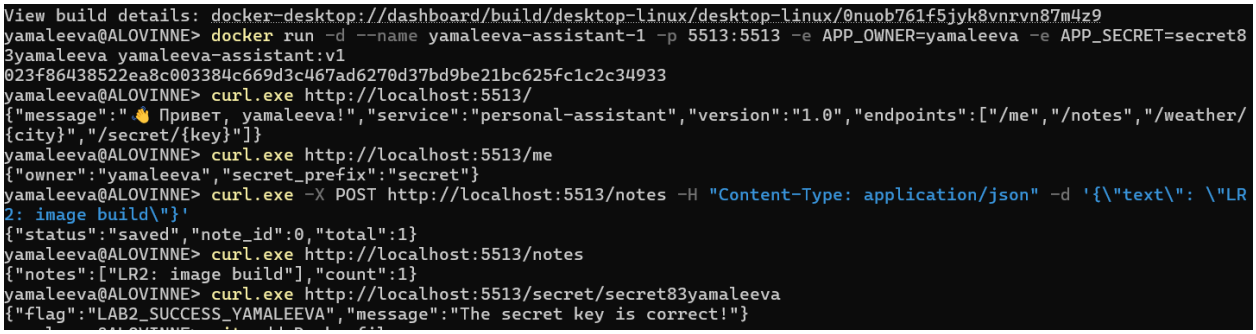



Рисунок 4 – Запуск контейнера и проверка эндпоинтов

# 2 ОПТИМИЗАЦИЯ ОБРАЗА

## 2.1 Оптимизированный Dockerfile

В Dockerfile.optimized зависимости по-прежнему копируются отдельно от кода, а при установке добавлен флаг --no-cache-dir, чтобы кэш pip не хранился в образе (рисунок 5).

```
yamaleeva@ALOVINNE> $u = New-Object System.Text.UTF8Encoding $false
yamaleeva@ALOVINNE> [System.IO.File]::WriteAllText("$PWD\Dockerfile.optimized", @'
>> FROM python:3.11-slim
>> LABEL maintainer="yamaleeva@lab.local"
>> LABEL version="1.0"
>> WORKDIR /app
>> COPY requirements.txt .
>> RUN pip install --no-cache-dir -r requirements.txt
>> COPY app/ ./app/
>> EXPOSE 5513
>> CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "5513"]
>> '@, $u)
```

Рисунок 5 – Создание Dockerfile.optimized

## 2.2 Сборка и сравнение размеров

Образ v2 собран из Dockerfile.optimized (рисунок 6). Размер v1 составляет 285 МБ, v2 – 258 МБ, то есть образ уменьшился на 27 МБ (рисунок 7).

```
yamaleeva@ALOVINNE> docker build -t yamaleeva-assistant:v2 -f Dockerfile.optimized .
[+] Building 69.3s (10/10) FINISHED                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile.optimized                                        0.1s
 => => transferring dockerfile: 317B                                                                  0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim                                   1.4s
 => [internal] load .dockerignore                                                                     0.0s
 => => transferring context: 2B                                                                       0.0s
 => [1/5] FROM docker.io/library/python:3.11-slim@sha256:e88e9763f943ec1834f992a4b51e0f24500486803e8bc534e5767af9  0.1s
 => => resolve docker.io/library/python:3.11-slim@sha256:e88e9763f943ec1834f992a4b51e0f24500486803e8bc534e5767af9  0.1s
 => [internal] load build context                                                                     0.0s
 => => transferring context: 182B                                                                     0.0s
 => CACHED [2/5] WORKDIR /app                                                                         0.0s
 => CACHED [3/5] COPY requirements.txt .                                                              0.0s
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt                                         59.9s
 => [5/5] COPY app/ ./app/                                                                            0.1s
 => exporting to image                                                                                7.3s
 => => exporting layers                                                                               4.8s
 => => exporting manifest sha256:3b200b1ac0ecaad87f18ded26e69307b3ef1144bcc268502b35a7ccf15cf1b63     0.0s
 => => exporting config sha256:5cc73b1e74abb7def61bab8990ced6575d8385c6ed180a52e9775d885a0fd548       0.0s
 => => exporting attestation manifest sha256:ba8559cc4461c4b6baad527e837b4a19f11da8d0294913673f8f720cb6894205  0.1s
 => => exporting manifest list sha256:a947af5050c467090ff38683f2b5f0d80f8756a5fcd0254700bb4090680f493b  0.0s
 => => naming to docker.io/library/yamaleeva-assistant:v2                                             0.0s
 => => unpacking to docker.io/library/yamaleeva-assistant:v2                                          2.2s
```

Рисунок 6 – Сборка образа yamaleeva-assistant:v2

```
yamaleeva@ALOVINNE> docker images --filter "reference=yamaleeva-assistant"
REPOSITORY            TAG       IMAGE ID       CREATED          SIZE
yamaleeva-assistant   v2        a947af5050c4   37 seconds ago   258MB
yamaleeva-assistant   v1        493fef9a698e   9 minutes ago    285MB
```

Рисунок 7 – Размеры образов v1 и v2

**Почему копирование requirements.txt отдельно от кода ускоряет повторные сборки?**

Docker кэширует каждый слой. Если requirements.txt не менялся, слой с pip install берётся из кэша и зависимости заново не устанавливаются. При изменении только кода пересобирается лишь последний слой с COPY app/.

### 2.3 Multi-stage сборка

Дополнительно создан Dockerfile.multistage. В первом этапе (builder) зависимости устанавливаются в папку /install, во втором в чистый образ копируются только эти зависимости и код приложения (рисунок 8, листинг 3). Образ собран командой docker build -t yamaleeva-assistant:multistage (рисунок 9). Его размер 260 МБ, что на 17 МБ меньше образа final (277 МБ), в котором тоже установлен curl (рисунок 10).

```
yamaleeva@ALOVINNE> [System.IO.File]::WriteAllText("$PWD\Dockerfile.multistage", @'
>> FROM python:3.11-slim AS builder
>> WORKDIR /app
>> COPY requirements.txt .
>> RUN pip install --no-cache-dir --prefix=/install -r requirements.txt
>>
>>
>> FROM python:3.11-slim
>> LABEL maintainer="yamaleeva@lab.local"
>> LABEL version="1.0"
>> WORKDIR /app
>> COPY --from=builder /install /usr/local
>> COPY app/ ./app/
>> EXPOSE 5513
>> RUN apt-get update && apt-get install -y --no-install-recommends curl && rm -rf /var/lib/apt/lists/*
>> HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 CMD curl -f http://localhost:5513/health || ex
it 1
>> CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "5513"]
>> '@, $u)
```

Рисунок 8 – Создание Dockerfile.multistage

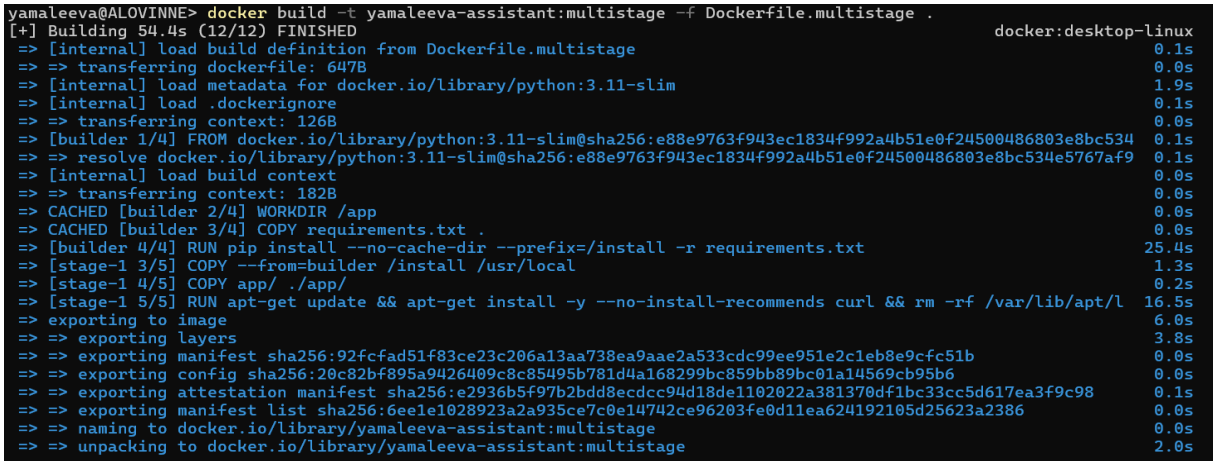



Рисунок 9 – Сборка образа yamaleeva-assistant:multistage

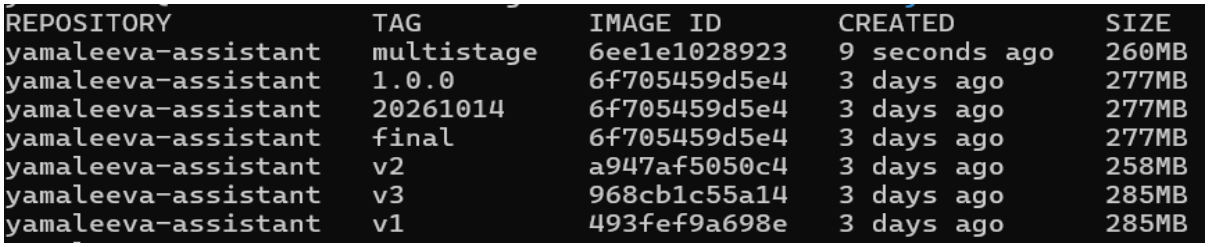



Рисунок 10 – Размеры образов, включая multistage

# 3 ЧИСТОТА ОБРАЗА

## 3.1 Создание .dockerignore

Создан файл .dockerignore. Он исключает из контекста сборки кэш Python (__pycache__, .pyc, .pyo), служебные файлы git (.git, .gitignore), виртуальные окружения (.venv, venv), файл .env, логи, Dockerfile и README.md (рисунок 11, листинг 2).

```
yamaleeva@ALOVINNE> $u = New-Object System.Text.UTF8Encoding $false
yamaleeva@ALOVINNE> [System.IO.File]::WriteAllText("$PWD\.dockerignore", @'
>> __pycache__/
>> *.pyc
>> *.pyo
>> .git
>> .gitignore
>> .venv/
>> venv/
>> .env
>> *.log
>> Dockerfile*
>> README.md
>> '@, $u)
```

Рисунок 11 – Создание .dockerignore

## 3.2 Пересборка образа

Образ v3 пересобран командой docker build -t yamaleeva-assistant:v3 . (рисунок 12). Размер передаваемого контекста сборки уменьшился с 11,02 КБ (у v1) до 126 Б, так как каталог .git больше не передаётся в сборку. Размер образа не изменился (285 МБ): Dockerfile копирует в образ только requirements.txt и app/.

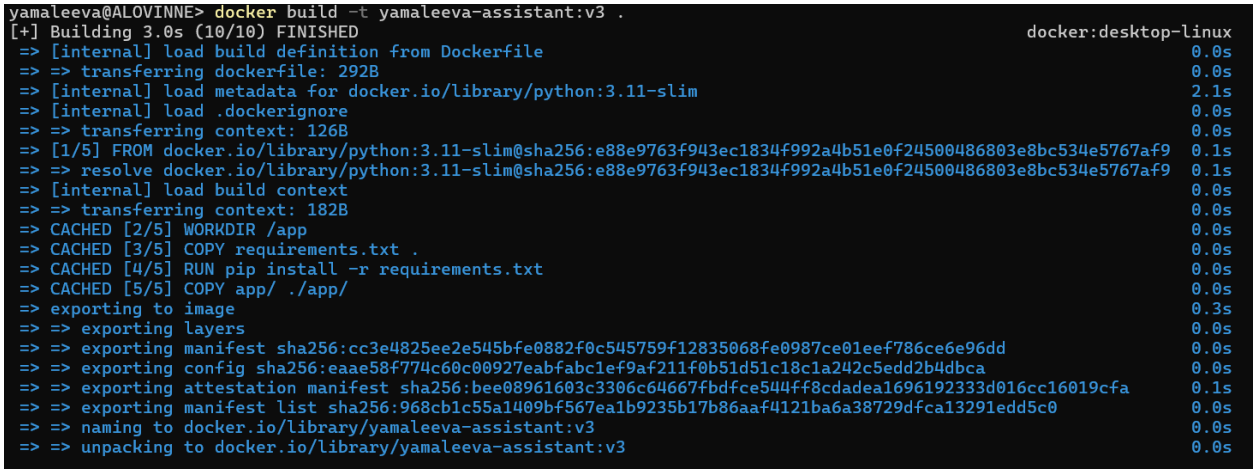



Рисунок 12 – Сборка образа yamaleeva-assistant:v3

**Какие файлы могли попасть в образ без .dockerignore?**

Кэш __pycache__ и файлы .pyc, каталог .git, виртуальное окружение, файл .env с секретами, логи, README.md и Dockerfile. В нашем Dockerfile копируется только app/, поэтому эти файлы попали бы в образ при использовании команды COPY . .

**Почему наличие .git в образе – потенциальная уязвимость?**

В .git хранится вся история проекта, включая старые коммиты. Если в них когда-либо были пароли, ключи или токены, их можно извлечь из образа, даже если в текущей версии файлов их уже нет.

# 4 ПРОВЕРКА ЗДОРОВЬЯ

## 4.1 Добавление healthcheck

В Dockerfile.optimized добавлены установка curl (в образе python:3.11-slim его нет) и инструкция HEALTHCHECK. Проверка выполняется каждые 30 секунд, таймаут 5 секунд, на запуск даётся 10 секунд, сервис считается нездоровым после 3 неудачных попыток. Проверяется адрес /health (рисунок 13). Полный текст файла приведён в листинге 1.

```
yamaleeva@ALOVINNE> $u = New-Object System.Text.UTF8Encoding $false
yamaleeva@ALOVINNE> [System.IO.File]::WriteAllText("$PWD\Dockerfile.optimized", @'
>> FROM python:3.11-slim
>> LABEL maintainer="yamaleeva@lab.local"
>> LABEL version="1.0"
>> WORKDIR /app
>> COPY requirements.txt .
>> RUN pip install --no-cache-dir -r requirements.txt
>> COPY app/ ./app/
>> EXPOSE 5513
>> RUN apt-get update && apt-get install -y --no-install-recommends curl && rm -rf /var/lib/apt/lists/*
>> HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 CMD curl -f http://localhost:5513/health || ex
it 1
>> CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "5513"]
>> '@, $u)
```

Рисунок 13 – Dockerfile.optimized с проверкой здоровья

## 4.2 Сборка и проверка статуса

Образ final собран из Dockerfile.optimized (рисунок 14). Первый контейнер перед этим удалён, так как он занимал порт 5513. Новый контейнер запущен, через 15 секунд его статус Up (healthy) (рисунок 15).

```
yamaleeva@ALOVINNE> docker rm -f yamaleeva-assistant-1
yamaleeva-assistant-1
yamaleeva@ALOVINNE> docker build -t yamaleeva-assistant:final -f Dockerfile.optimized .
[+] Building 24.1s (11/11) FINISHED                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile.optimized                                                        0.1s
 => => transferring dockerfile: 540B                                                                                  0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim                                                   4.8s
 => [internal] load .dockerignore                                                                                     0.0s
 => => transferring context: 126B                                                                                     0.0s
 => [1/6] FROM docker.io/library/python:3.11-slim@sha256:e88e9763f943ec1834f992a4b51e0f24500486803e8bc534e5767af9     0.1s
 => => resolve docker.io/library/python:3.11-slim@sha256:e88e9763f943ec1834f992a4b51e0f24500486803e8bc534e5767af9     0.1s
 => [internal] load build context                                                                                     0.0s
 => => transferring context: 182B                                                                                     0.0s
 => CACHED [2/6] WORKDIR /app                                                                                         0.0s
 => CACHED [3/6] COPY requirements.txt .                                                                              0.0s
 => CACHED [4/6] RUN pip install --no-cache-dir -r requirements.txt                                                   0.0s
 => CACHED [5/6] COPY app/ ./app/                                                                                     0.0s
 => [6/6] RUN apt-get update && apt-get install -y --no-install-recommends curl && rm -rf /var/lib/apt/lists/*      16.4s
 => exporting to image                                                                                                2.2s
 => => exporting layers                                                                                               1.5s
 => => exporting manifest sha256:62e3992041c67147c75af805802098fc1b25fda36c1d7da2f50ee064f4286b50                     0.0s
 => => exporting config sha256:3219e9157aad222ec4e23fda8321a2aeda5a6a3eb05528bfe5279ba73a8af924                       0.0s
 => => exporting attestation manifest sha256:a09acd224d5e778568a4d4278099771fc91ba917518d03753f9503c24d13324c        0.1s
 => => exporting manifest list sha256:6f705459d5e4e5f16bd0b367b371de79194a42800f1abf96366cb1349c7ecd2c                0.0s
 => => naming to docker.io/library/yamaleeva-assistant:final                                                          0.0s
 => => unpacking to docker.io/library/yamaleeva-assistant:final                                                       0.4s
```

Рисунок 14 – Сборка образа yamaleeva-assistant:final

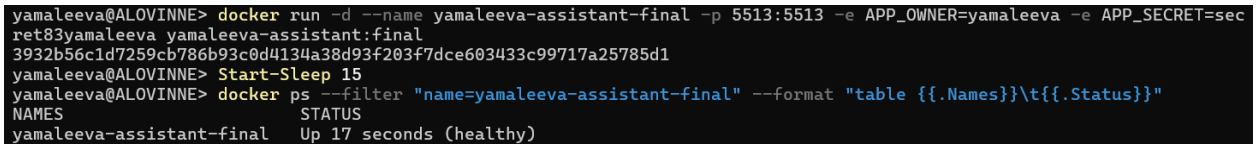



Рисунок 15 – Запуск контейнера и статус healthy

# 5 ФИНАЛЬНЫЙ АРТЕФАКТ И ТЕГИРОВАНИЕ

Образу final присвоены теги 1.0.0 и 20261014 (дата сборки). Все теги указывают на один образ (ID 6f705459d5e4) размером 277 МБ. Вывод финальной команды с логином, хостом и временем приведён на рисунке 16.

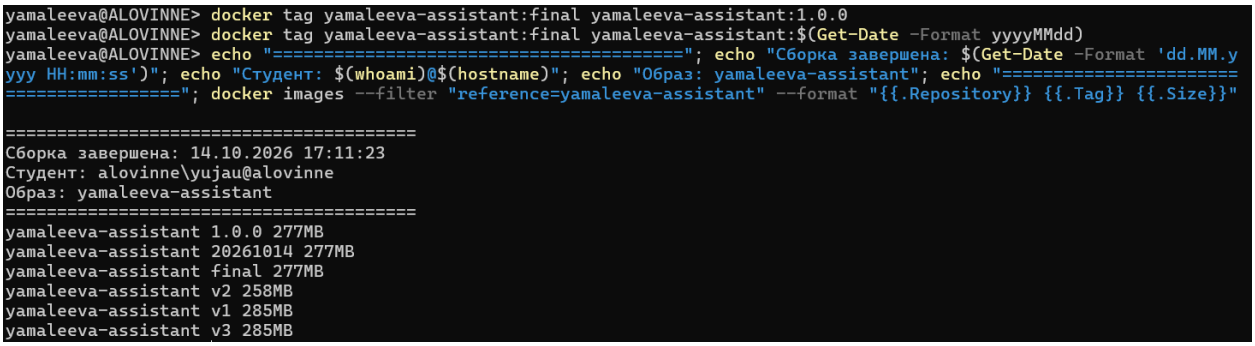



Рисунок 16 – Финальный артефакт выполнения

## ЗАКЛЮЧЕНИЕ

В ходе работы собран образ приложения assist-ру из Dockerfile, контейнер запущен с переменными окружения и проверен запросами к эндпоинтам. Оптимизация (--no-cache-dir) уменьшила образ с 285 до 258 МБ, multi-stage сборка дала образ 260 МБ вместе с curl.

С помощью .dockerignore из контекста сборки исключены лишние файлы (контекст уменьшился с 11,02 КБ до 126 Б). Добавлен healthcheck, контейнер получил статус healthy. Образу присвоены семантические теги: 1.0.0 и по дате сборки.